Planificador de visitas

Prueba técnica Talent Clau · Resumen técnico · 31/10/2026

Implementación

Agenda para cuatro técnicos y cuatro obras ficticias. Permite crear visitas con fecha, hora y duración; consultar y combinar filtros por técnico/fecha; y cancelar conservando el registro y liberando su horario. El servidor explica los rechazos por solapamiento, desplazamiento insuficiente o jornada. El listado muestra fecha, intervalo, técnico, obra, municipio y estado, ordenados cronológicamente.

Decisiones y supuestos

Una aplicación Next.js, React y TypeScript, con PostgreSQL en Neon Free, Drizzle para consultas/migraciones y Zod para entradas. Un solo proyecto reduce la configuración. Las reglas independientes de la interfaz facilitan probar y explicar cada restricción. Cada escritura bloquea la fila del técnico dentro de una transacción antes de consultar su agenda; así se evitan reservas incompatibles simultáneas, incluso con agenda vacía.

Se guardan fecha civil de Madrid, inicio en minutos y duración entera positiva. La visita completa debe estar entre 08:00 y 17:00. El final del intervalo se excluye: dos visitas consecutivas en la misma obra son válidas. Obras con distinto ID requieren 30 minutos, también en el mismo municipio y respecto a visitas anteriores y posteriores. Las canceladas no bloquean. No se añaden restricciones de festivos, fines de semana, pasado ni comida.

Verificación y pendientes

Superadas 71 pruebas unitarias y 21 de integración con PostgreSQL, incluidas restricciones, persistencia, filtros y concurrencia desde conexiones independientes. Tipos, lint y compilación correctos. Flujos principales comprobados en navegador de escritorio. Arranque desde copia limpia en 18,66 s con Node, red y PostgreSQL disponibles; instrucciones en README. Migración y semilla reproducibles.

Pendientes: validación personal del usuario, revisión visual móvil y publicación del repositorio público. Despliegue y sugerencias de horarios son opcionales y no se han implementado. Es una demostración sin autenticación; todas las escrituras deben pasar por el servicio para conservar las reglas entre visitas.

Uso de IA y corrección real

Codex ayudó con análisis, código, pruebas y documentación; el usuario aprobó alcance, arquitectura y supuestos. La revisión en navegador detectó un fallo generado por IA: se comparaba el origen con la URL interna de Next.js, que convertía 127.0.0.1 en localhost y rechazaba creaciones legítimas con HTTP 403. Se reprodujo en REG-001, se corrigió usando el encabezado Host para la dirección solicitada y se verificó tanto la aceptación legítima como el rechazo de otros orígenes. Después se repitieron integración, compilación y el flujo real de creación/cancelación. No se inventaron errores para esta evidencia.